IMD3002 – 3D COMPUTER GRAPHICS

Prof. Chris Joslin

L20 – Rendering: Introduction

Overview

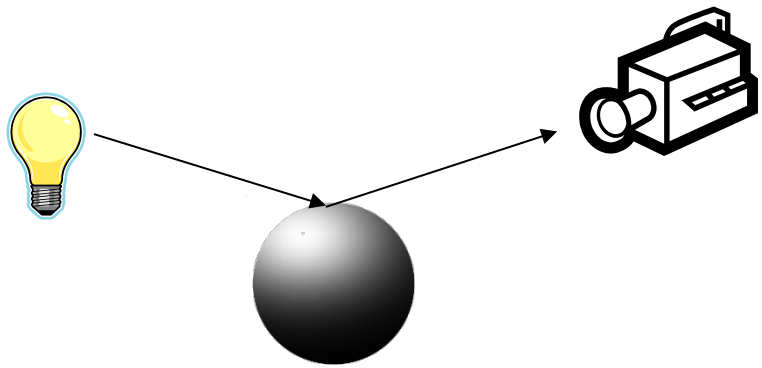
- This lecture covers more advanced rendering techniques under the umbrella of Global Illumination

Introduction

- Direct Illumination
  - Earlier in the course Direct Illumination models (sometimes called Local Illumination) were introduced
  - They project 3D models into a canonical view volume (CVV) and then determine the pixel colour through a visibility algorithm (such as painter's algorithm, scan-line rendering, or z-buffering)

Introduction

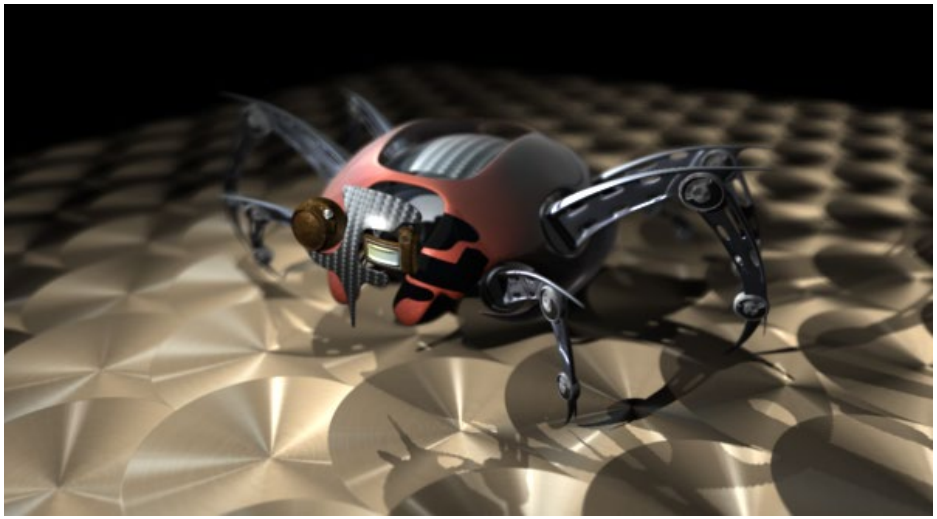
- Direct Illumination
  - The Direct Illumination model uses a single bounce to determine a polygon's colour
  - This is essentially a light to polygon, polygon to camera bounce



  - Because there is only a single bounce, shading is simple, and shadows are not calculated directly (and are added as an additional method)

Introduction

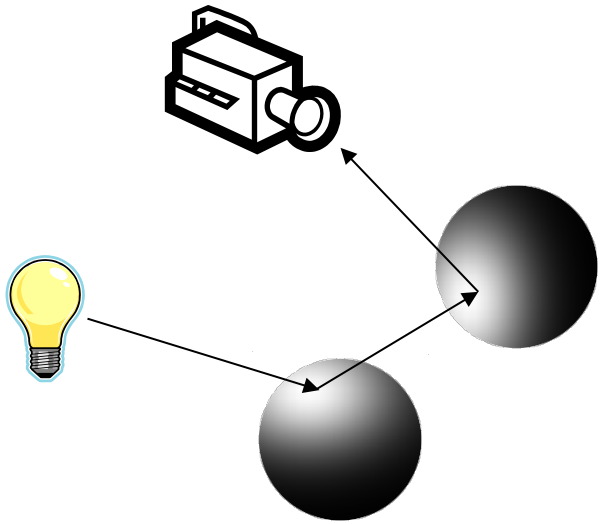
- Direct Illumination
  - There are many rendering engines that use Direct Illumination by default
    - **Examples:** OpenGL and DirectX
  - Direct Illumination is fast and generally produces acceptable, if not fairly realistic, renders



This is rendered using OpenGL, but has additional features on top (such as shadows)

Introduction

- Global Illumination
  - The term “Global Illumination” (GI) umbrella’s many different types of rendering systems, the main criteria of which is that more than one light bounce is taken into account



Global Illumination

- Rendering Methods
  - There are many different GI methods available, some are focused on realism and others on speed
  - Some are more popular than others (as it's an active research field):
    - Ray Tracing (Forwards & Backwards)
    - Path Tracing
    - Radiosity Rendering
    - Photon Mapping
    - Ambient Occlusion
    - Cone/Beam Tracing
    - Final Gather

Global Illumination

- Rendering Systems
  - There are many popular rendering systems that use GI, they are often offline (non-real-time) rendering systems:

- Mental Ray (Mental Images/NVidia)
  - Star Wars, Poseidon, Spiderman, The Matrix
- Arnold (Solid Angle)
  - Gravity, Pacific Rim, Elysium
- Final Render (Cebas)
  - Whitehouse Down, Hellboy II
- V-Ray (Chaos Group)
  - Architectural
  - Game of Thrones, Oblivion, Cloud Atlas

- Maxwell Renderer (NextLimit)
- RenderMan (Pixar)
  - All Pixar Movies
- Octane (GPU Driven)
- KeyShot
- Pixie (Open Source RenderMan)
  - Rendering examples only
- POV-Ray (Open Source)
  - Rendering examples only
- Brazil (no longer available)

Global Illumination

- Rendering Systems
  - The choice of renderer can depend on several factors:
    - **Quality** ~ different methods, and even the application of the same methods, can yield different quality levels
    - **Time** ~ rendering time for any type of GI tends to be huge (from minutes to hours/frame)
      - Therefore, the faster a rendering system can process a frame, with the same quality, the better
    - **Cost** ~ due to the huge amount of effort it takes to perfect a GI rendering system, some can cost upwards of $1000 per license (i.e. per node)
      - Mental Ray with Maya, RenderMan is free for academia

Global Illumination

- Rendering Systems
  - Choice of Renderer (cont'd):
    - **Functionality** ~ some rendering systems are only basic ray tracers and offer realistic rendering with few functions, others enable us to select everything from the type of lens, to colour temperature of the light
    - **Ease** ~ all GI systems have many configuration settings, and just setting some of the more basic functionality is difficult; however some systems have simplified this process
  - Evaluate as many different systems before committing to a final choice

Global Illumination

- The Rendering Equation
  - Most modern GI renderers are based on what is known as the "Rendering Equation", from James T. Kajiya (1986, The Rendering Equation)
  - This is an approximation of Maxwell's equation for electromagnetics; it does not model all optical phenomena, only geometric optics
    - It cannot simulate diffraction, interference, or polarisation
    - It is able to be modified to model dispersion
  - The introduction of the rendering equation was coupled with a method called "Path Tracing" which we shall discuss in the next lecture

Global Illumination

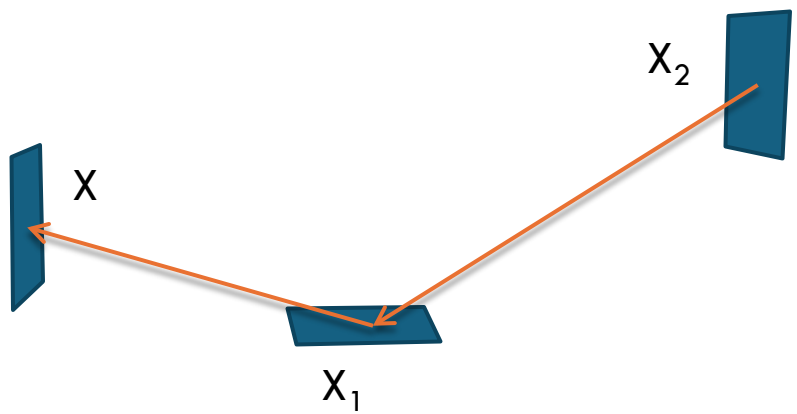
- The Rendering Equation
  - It should be noted that the rendering equation is mathematically modelling the way light behaves, it is a very good model, but not exact
  - The equation is as follows:

$$L(x, x_1) = g(x, x_1)[e(x, x_1) + \int_S r(x, x_1, x_2) L(x_1, x_2) dx_2]$$

- $L(x,x_1)$ is related to the light passing from point x_1 to point x
- $g(x,x_1)$ is a geometry (or visibility term)
- $e(x,x_1)$ is the intensity of emitted light from point x_1 towards point x
- $r(x,x_1,x_2)$ is related to the light scattered from point x_2 to point x, through x_1
- S is the union of all surfaces in the scene and x, x_1, and x_2 are points from S

Global Illumination

- The Rendering Equation
  - What the equation means: the light arriving at a given point x in the scene from another point x_1 is the sum of the light emitted from all other points x_2 towards x_1 and reflected towards x



Global Illumination

- The Rendering Equation
  - Except for very simple cases, the rendering equation cannot be solved exactly in a finite amount of time on a computer
  - We can get as close as we want to the real solution - given enough time
  - The search for global illumination algorithms has been a quest for finding solutions that are reasonably close, for a reasonable amount of time
  - Different renderers apply different methods for solving it

Global Illumination

- The Rendering Equation
  - If any two renderers solve this equation accurately enough, then they should generate the same image for the same scene
  - This is very well in theory, but in practice renderers often truncate or alter parts of the rendering equation, which may lead to different results

Global Illumination

- Basic Method
  - Most of these methods are approximately the same, they compute the path of a ray (either as a line, a beam, cone, etc.) from source to destination reflected (and refracted) many times from light to camera (and often in reverse)
  - We shall look in detail at several key methods and then briefly mention the others for completeness as they are really specialised and currently not very popular

Ray Tracing

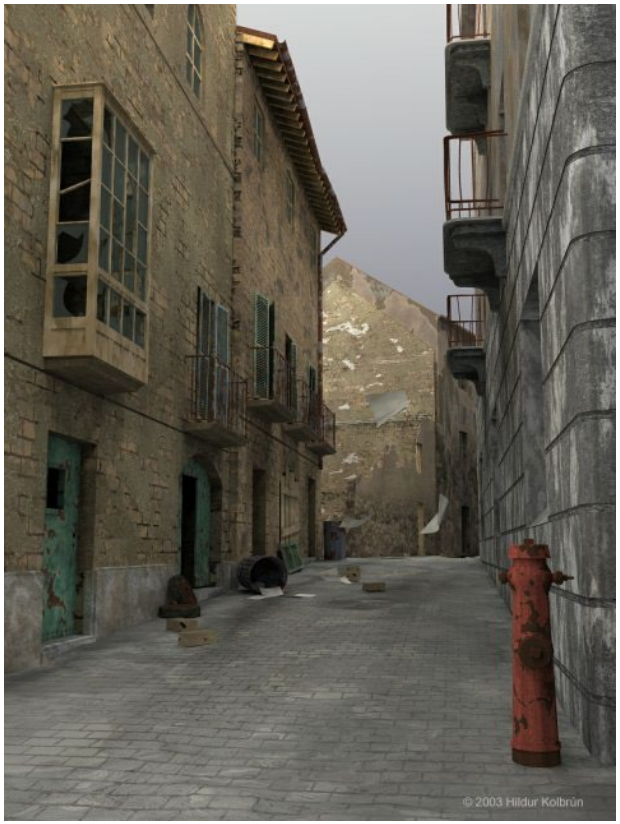
- Introduction
  - The term “Ray Tracing” is used because the method tries to simulate the path that light rays take as they bounce around within the world - they are effectively *traced* through the scene
  - The objective is to determine the colour of each light ray that enters the view window, which is used to determine the colour of a pixel in the viewport
  - A light ray can best be thought of as a single photon (although this is not entirely accurate either, but it’s close enough)

Ray Tracing

- Introduction
  - **Examples:** Ray Traced renderings (from PoVRay)

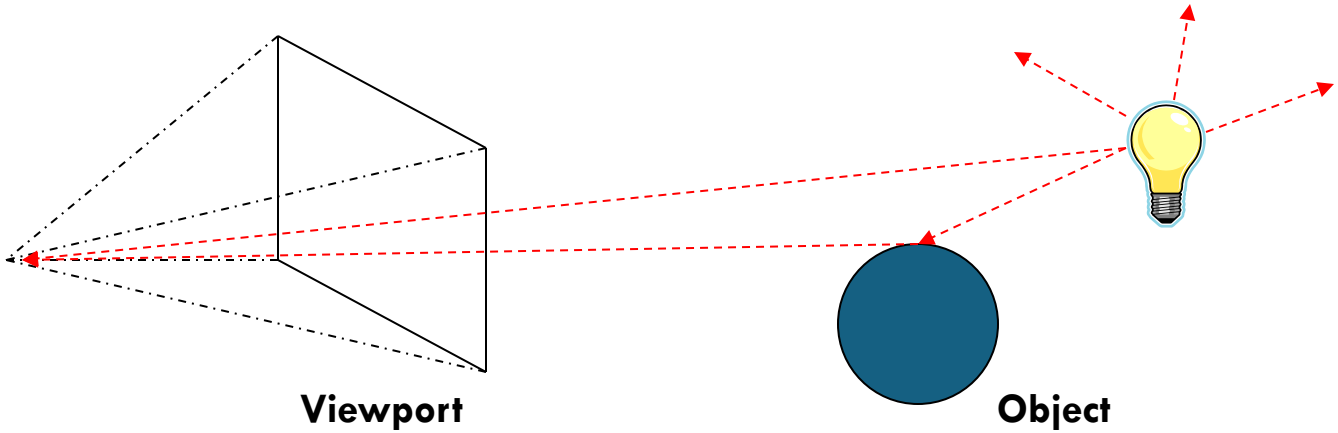
Ray Tracing

- Introduction
  - **Examples:** Ray Traced renderings (from PoVRay)



Ray Tracing

- Introduction
  - A physically-based lighting equation will trace a light ray from the source to the camera
  - However, the problem is that it is not known whether a light ray will actually reach the camera until the calculation is made – this means lots of wasted calculations



Ray Tracing

- Introduction
  - Calculations are wasted because many rays never reach the camera: they might shoot off into space (or the stratosphere), or they may become so attenuated due to the inverse square law for energy distribution ($1/d^2$) that they make little contribution to the overall illumination of the scene
  - Therefore, in order make a more efficient use of the available processing power, only rays that are sure to reach the viewport are traced

Ray Tracing

- Introduction
  - This sounds like another impossible task, however turning this around and tracing the ray from the view back to the light, there is a guarantee that the ray either reaches a light or disappears off into space (and hence it is black)
  - This is the key method of Ray Tracing and it often referred to as "Forward Ray Tracing"
    - This is a little strange as it is working backwards, but this comes from the history of ray tracing that this method was used first, and then other methods working in reverse (sometimes called "Backwards Ray Tracing")

Ray Tracing

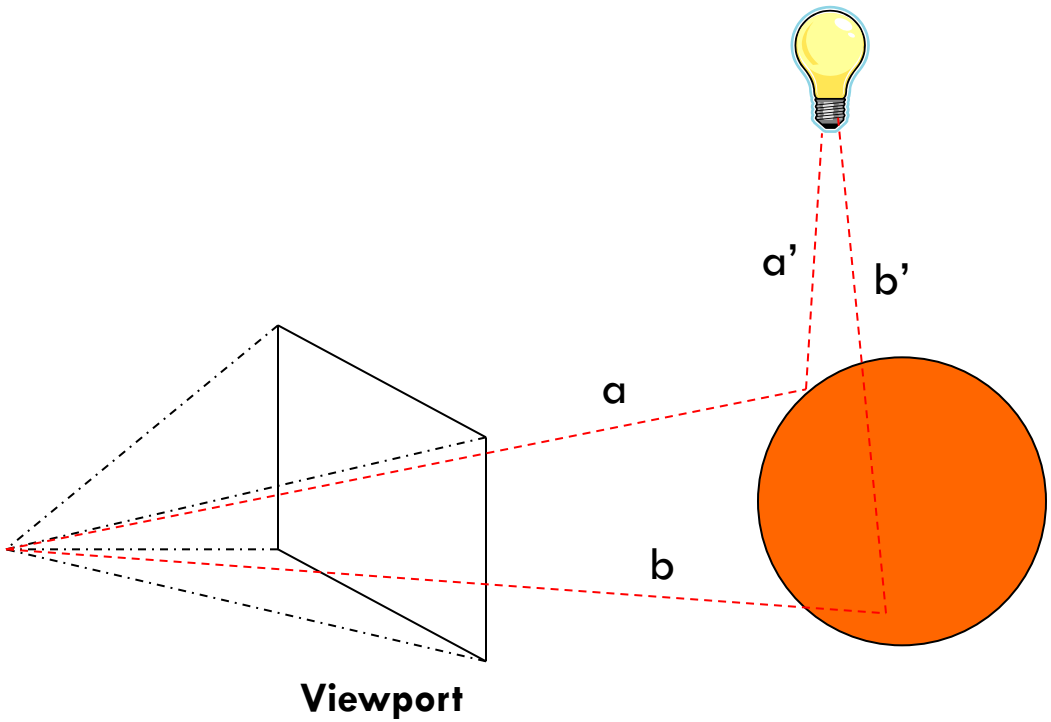
- Introduction
  - For each pixel in the viewport a ray is defined that extends from the frustum apex (or eye position)
  - This ray is sent out into the scene and traced as it bounces off different objects in the scene
  - The final colour of the ray (and thus the colour of the pixel it intersects) is given by an accumulation of the colour of the objects hit by the ray as it makes its way through the scene

Ray Tracing

- Source and Shadows
  - In a hypothetical ideal renderer where the “light-to-camera” ray would require many reflections before it made its way to the camera, the ray tracing method can also have this overhead
  - Since there is a need to establish a limit on the number of bounces for the ray, an approximation is made
  - Each time a ray hits an object, a new ray is created from the point of intersection directly to the light source

Ray Tracing

- Source and Shadows



Ray Tracing

- Source and Shadows
  - In the previous diagram we have two rays, **a** and **b**, which intersect the orange sphere
  - In order to determine the colour of **a**, the new ray **a'** is followed directly (i.e. no more reflections) to the source
  - The point where the ray **b** intersects is an area in shadow, as **b'** intersects the sphere
  - Ray **a** would be also in shadow if another object was intersected by ray **a'** before it reached the light

Ray Tracing

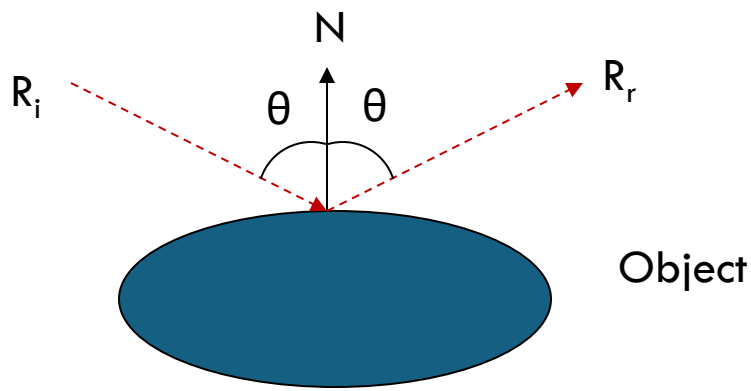
- Reflection & Refraction
  - To handle **Reflections** we need to consider multiple bounces from objects
  - To handle **Refractions** we need to consider what happens when a ray passes through a partially or fully transparent object
  - If an object is reflective, we simply trace a new reflected ray from the point of intersection towards the direction of reflection
  - The reflected light ray is the mirror image of the original light ray, pointing away from the surface

Ray Tracing

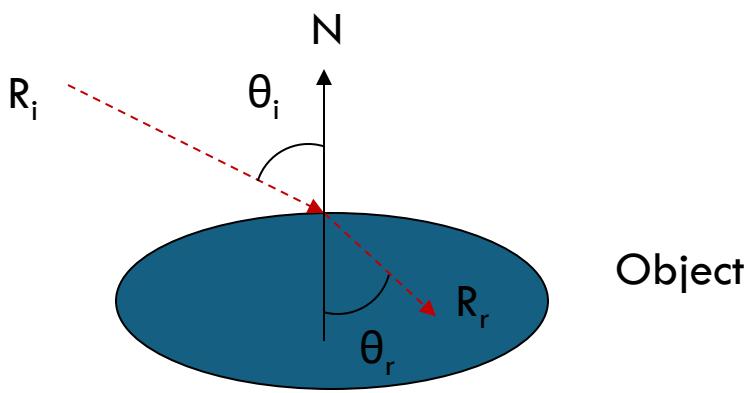
- Reflection & Refraction
  - If the object is at least partially transparent, then a refracted ray is traced into the surface
  - If there are differences in the "indices of refraction", (if this information is known) then the refracted ray will be refracted
    - **Example:** air on one side and water on the other, makes a straight pole appear bent if it is sticking out the water
  - If the same medium exists on both sides, then the ray travels in the same direction as the original ray, and is not bent

Ray Tracing

- Reflection & Refraction
  - Reflected Ray



  - Refracted Ray



Ray Tracing

- Reflection & Refraction
  - We can easily calculate the vectors of the reflected rays using the following equation:

$$R_r = R_i + (2 \times N \times -(N \cdot R_i))$$

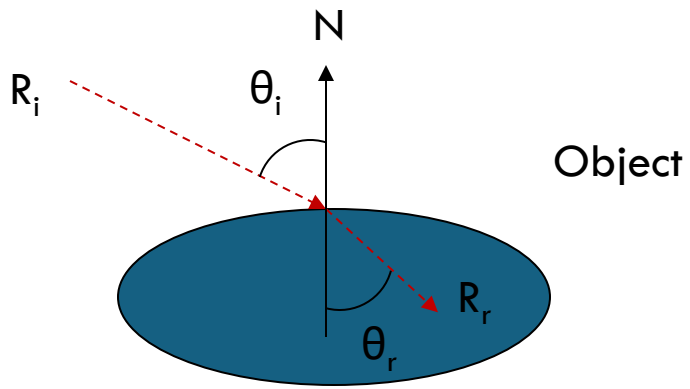
  - Where:
    - R_i ~ Incident Ray
    - R_r ~ Reflected Ray
    - N ~ Normal Vector

Here we have again the Dot Product between two vectors a vector/scalar multiplication, and then a vector addition

We also see the Normal vector is useful for ray tracing as well

Ray Tracing

- Reflection & Refraction
  - Refracted Ray



Ray Tracing

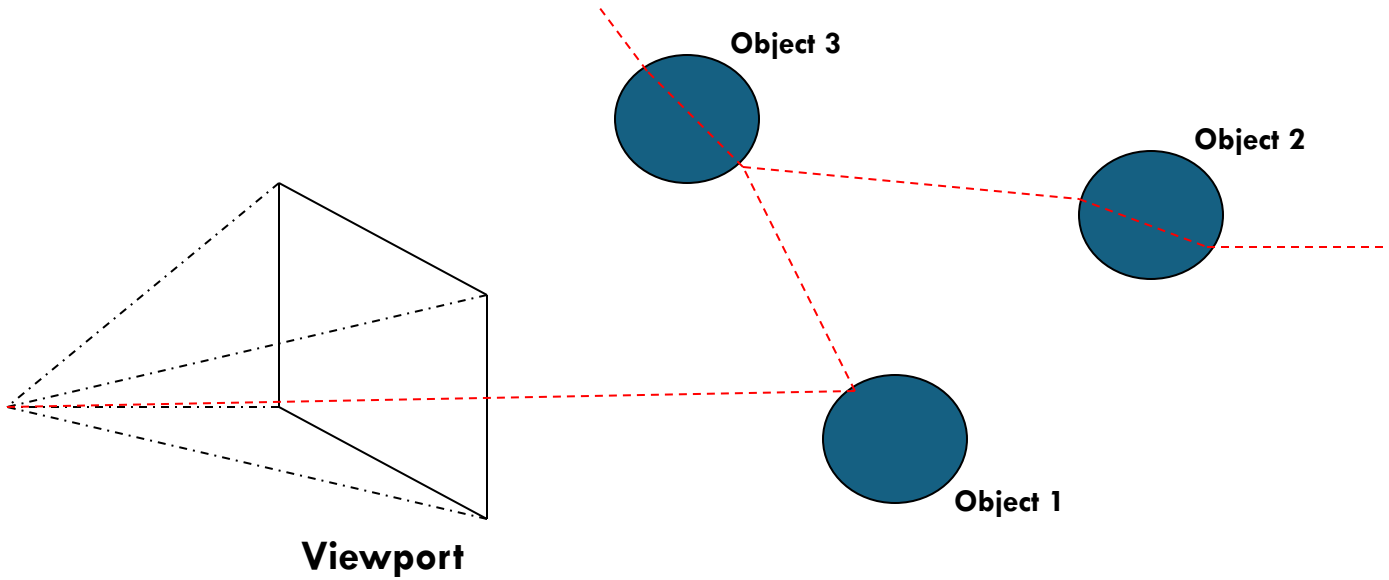
- Reflection & Refraction
  - To calculate the refracted ray vector (R_r), the following equation is used:

$$n = n_1 / n_2$$
$$c_1 = -(N \cdot R_i)$$
$$c_2 = \sqrt{1 - n^2 \times (1 - c_1^{\,2})}$$
$$R_r = (n \times R_i) + (n \times c_1 - c_2) \times N$$

  - Where:
    - n_1, n_2 - Index of refraction (prior/post)
    - R_i - Incident Ray
    - R_r - Refracted Ray
    - N - Normal Vector

Ray Tracing

- Reflection & Refraction
  - If we look how reflection and refractions can continue based on the laws and equations have seen previously, things start to become very complicated



Ray Tracing

- Reflection & Refraction
  - Now each ray can strike an object, which then spawns another two rays, each of these may spawn another two, and so on
  - This arrangement of rays spawning sub-rays leads to the idea of a "**tree of rays**"
  - The root node is the original ray from the eye, and each node in the tree is either a reflected or refracted ray resulting from the ray (node) above it
  - This tree structure is simply a recursive function and very simple to implement

Ray Tracing

- Intersections
  - One of the basic computations that a ray tracer needs to make is testing intersection for each object in the scene
  - If a ray intersects an object, the intersection calculation returns the distance of the intersection point from the origin of the ray, the normal vector at the point of intersection, and if texture mapping is being used – a coordinate mapping between the intersection point and the intersection point

Ray Tracing

- Intersections
  - The distance to the intersection point is needed due to the fact that if a ray intersects more than one object, the closest intersection point is selected to confirm the occluding object
  - The normal vector is used to determine the shade at that specific point, since it describes in which direction the surface is facing and therefore affects how much light the point receives

Ray Tracing

- Intersections
  - This intersection process makes the computation for hidden surfaces, and the use of a z-buffer redundant as it is implied in this case
  - In some cases, a z-buffer may still be used to provide quick redrawing effects after a rendering is completed

Ray Tracing

- Colour and Shading
  - There are many different ways to determine the colour at a point of intersection
    - Some methods are very simple (e.g. flat shading) where every point on the object has the same colour
    - Other techniques, such as the "Cook-Torrance Model", are more complex, taking into account the physical attributes of the material in question
    - In general anything from the Lambert model to Phong Reflections can be used

Ray Tracing

- Advantages
  - Photorealistic features such as reflections, transparency, multiple lights, shadows, area lights, etc are easily achieved because rays are a good analogy for light paths, thereby modelling the real-world properties of light
  - Constructive Solid Geometry (CSG) modelling is easy to support
    - CSG is essentially a complex surface generated by combining simpler objects (through Boolean operations)
  - Motion Blur can be done easily, but generally requires multi-pass rendering

Ray Tracing

- Advantages
  - Ray tracing can make use of a hierarchical bounding boxes for scenes with larges amount of geometry, and because the ray/geometry intersection test is only performed on world-aligned bounding boxes, searches are relatively fast
    - Hierarchical bounding boxes are bounding volumes (think simple cuboid, i.e. 6 quads) that encompass more complex geometry; they are hierarchical as bigger cuboids can encompass smaller ones
    - From this a tree data structure is formed, which means that if a parent box doesn't intersect with a ray, then child branches will not (because the parent is bigger than the child)

Ray Tracing

- Advantages
  - Altering how the eye rays are projected into the scene can be used to imitate the optical properties of many different lenses, scene projections, and special lens distortions

Ray Tracing

- Performance Inhibitors
  - Although each screen pixel need only be computed once, that computation is expensive. This can occur even for pixels that are not intersected by any geometry
  - In scan-line rendering, there are no computations required for pixels that do not intersect geometry - the expensive operations are projecting the polygon into camera space, projecting each polygon into screen space, and computing per-polygon edge lists; the larger a polygon is, the more pixels it has

Ray Tracing

- Backwards Ray Tracing
  - Here we start with the scene, and cast rays in different directions until they strike a surface in the scene
  - At this point the ray tries to determine the amount of light available to illuminate this surface
  - This can happen with the help of an ambient term, which represents an (unrealistic) global brightness of the scene, by determining the distance to one or several light sources, or recursively by sending more rays into the scene from that point on

Ray Tracing

- Backwards Ray Tracing
  - Whilst backwards Ray Tracing sounds like what we have just discussed, it is in fact the reverse
  - Due to this confusion, you may also find that the terms "Eye-based" and "Light-based" are used to refer to the classical and its inverse respectively

Photon Mapping

- Introduction
  - In Photon Mapping, rays from all light sources and rays from the camera are traced independently until they are terminated (too far, decayed too much, etc.)
  - The second step connects the two ray sets to produce a luminance value for each surface
  - This is used to realistically simulate the interaction of light with different objects

Photon Mapping

- Introduction
  - Photon Mapping is capable of simulating the refraction of light through transparent substances such as glass or water
  - Can diffuse inter-reflection between illuminated objects
  - Produce effects caused by particulate matter such as smoke or water vapor (which is useful when dealing with Particle Systems)

Photon Mapping

- Introduction
  - **Example:** images rendered using Photon Mapping

Photon Mapping

- Method
  - With photon mapping, light packets called photons are sent out into the scene from the light source
  - Whenever a photon intersects with a surface, the intersection point, incoming direction, and energy of the photon are stored in a cache called the "***photon map***"
  - After intersecting the surface, a new direction for the photon is selected using the surface's bidirectional reflectance distribution function (BRDF)

Photon Mapping

- Method
  - There are two key methods for determining when a photon should stop its reflection
  - The first is to decrease the energy of the photon at each intersection point
  - When a photon's energy reaches a predetermined low-energy threshold, the reflections cease

Photon Mapping

- Method
  - The second method uses a "**Monte Carlo**" method technique called "**Russian Roulette**"
  - In this method, at each intersection the photon has a certain probability value of continuing to reflect inside the scene, and that likelihood decreases with each reflection

Radiosity

- Introduction
  - Radiosity is a GI method that tracks the spread of diffuse light around a scene
  - It attempts to simulate the effect of multiple light reflection
  - However, unlike basic ray tracing, which tracks only the specular transport of light, radiosity only tracks the diffuse transport of light

Radiosity

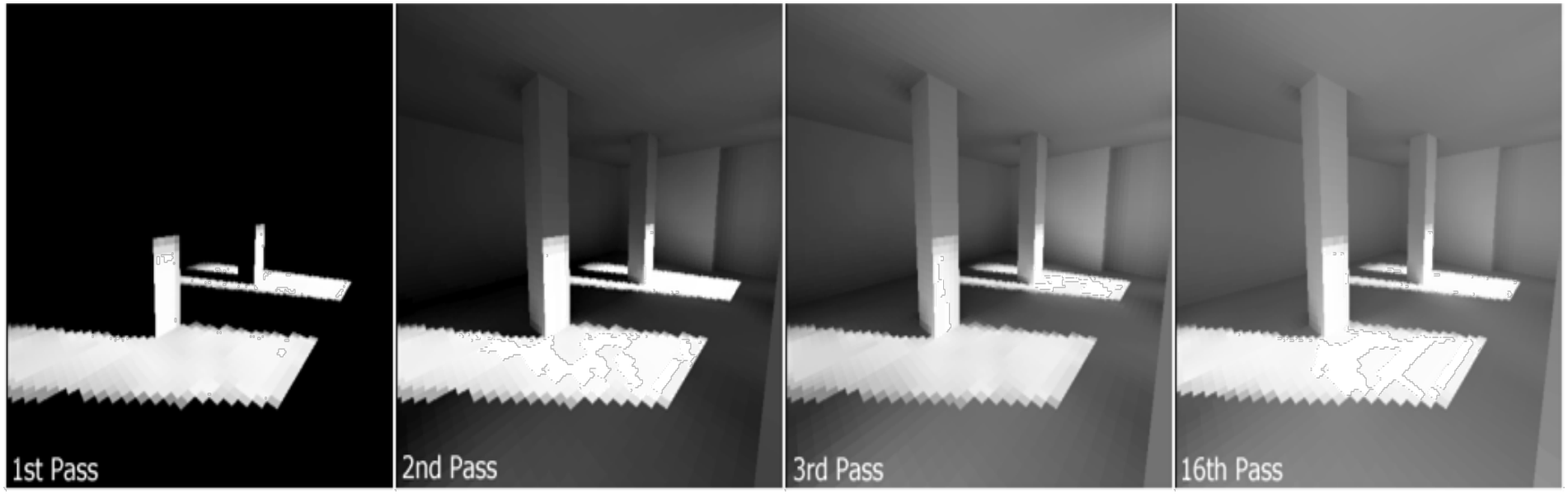
- Introduction
  - Radiosity is much better at softly lit scenes, with subtle colour shading – whereas Ray Tracing is much better at rendering sharp shadows
  - Radiosity methods usually track the diffusely reflected light by starting at the light source and tracking the light ray forward (which runs in the opposite to most ray tracers)

Radiosity

- Introduction
  - For this reason, the results of a ray tracing are view dependent (as expected from specular reflection)
  - However, we can use radiosity to pre-process a scene and once completed we can traverse the scene in real-time because there is no view dependency – which is ideal for walk-through applications
  - By comparison Ray Tracing needs to be repeated for each change in camera position, and therefore cannot be used for real-time

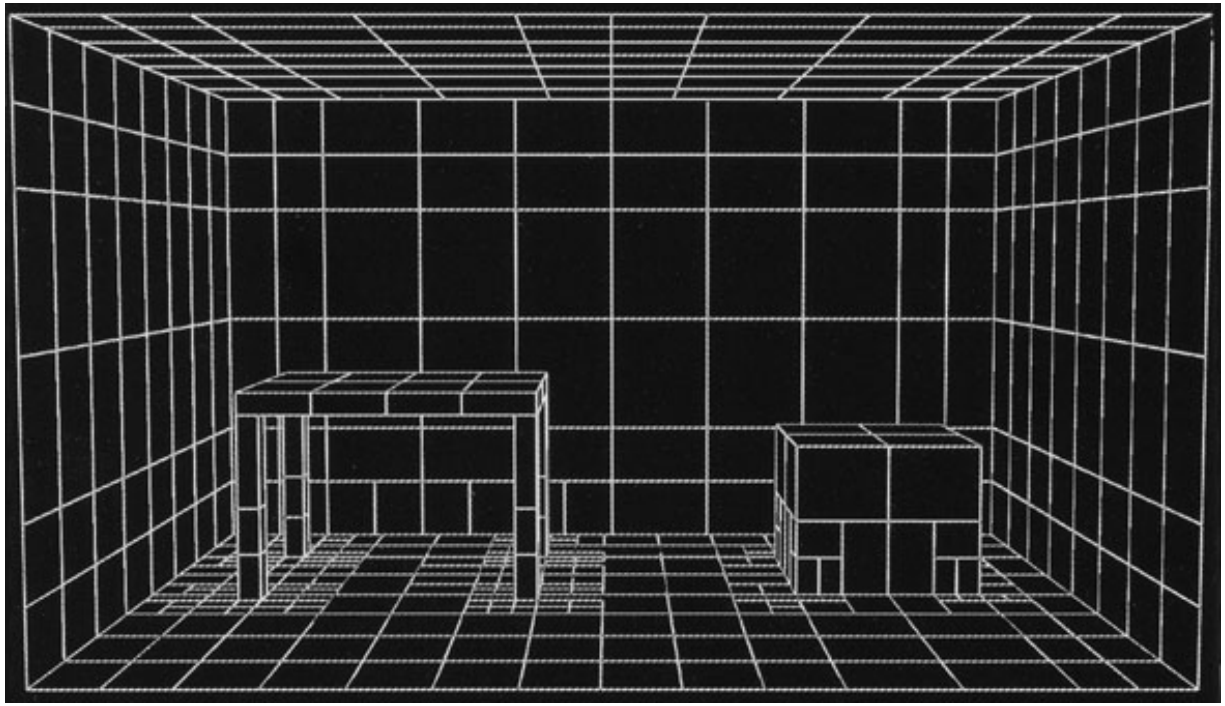
Radiosity

- Introduction
  - **Example:** each additional radiosity pass, increases the overall light



Radiosity

- Method
  - Radiosity algorithms typically begin by breaking down a scene into a set of flat polygons called "patches" as shown below



Radiosity

- Method
  - The main goal of the algorithm is to compute an illumination level, for each patch – for this the algorithm assumes that each patch is uniformly lit
  - It is assumed that all light is reflected only diffusely (which is where radiosity has a problem, it cannot handle highly reflective surfaces very well)
  - Therefore, it is only the total illumination of a patch which is important, and the direction of the incoming light can be ignored

Radiosity

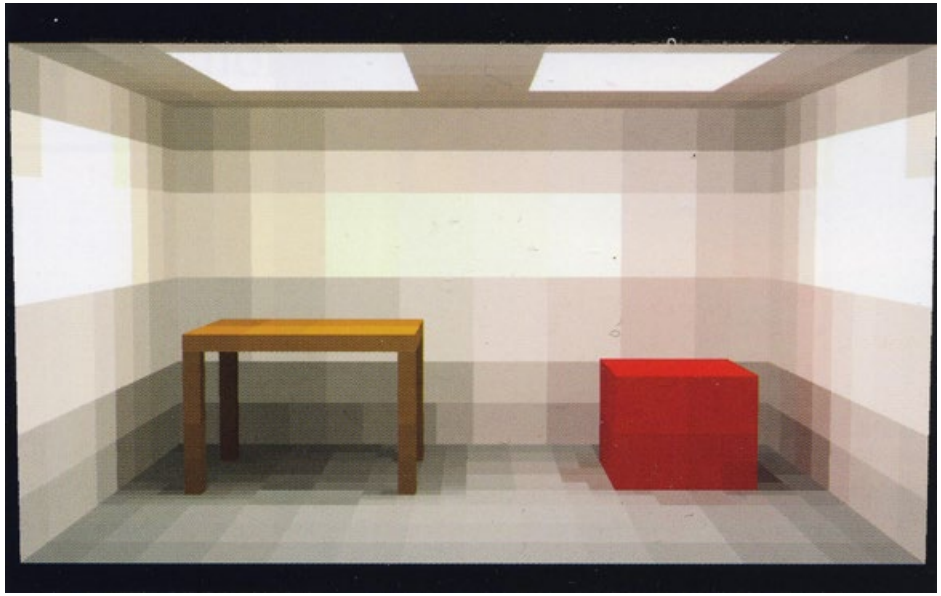
- Method
  - For most polygons (which are not emissive) we trace the flow of all light
  - This is done from the light source through multiple reflections in order to determine the overall light levels for each patch

Radiosity

- Method
  - After breaking the scene into patches, the radiosity algorithm computes “form factors”
  - These form factors are used to describe how much light reflects diffusely from one patch to another
  - For each pair of patches, their form factor measures the fraction of light leaving one patch and illuminating the next

Radiosity

- Method
  - Once the form factors have been determined, we set equations that relate the patch illuminations and the form factors
  - These equations can be conveniently setup as a large system of linear equations (that are nice to solve)
  - The equations used compute a single brightness for each patch, but we calculate Red, Green and Blue separately; however both form factors and patches are used for each channel

Radiosity

- Result
  - Here is the result of a scene separated into patches, we can see the colours of the patches are uniform, but they take the diffuse reflection colour from adjacent patches